Zomato Hackathon 2026

Problem Statement 2

Cart Super Add-On (CSAO) Rail Recommendation System

4.56ms Mean Inference Latency	**+5.7%** Projected AOV Lift	**0.705** AUC-ROC (Test Set)
7.01ms p99 Latency	**Rs.1,586 Cr** Annual Revenue Lift	**28x** Faster Than 200ms SLA

Team: **Binary Insights**

Members: Kunal Sharma, Anshika Tyagi

Submission Date: March 2, 2026

GitHub Repo: `https://github.com/your-repo/zomathon-csao`

Dataset: Reproduced via `phase1_data_generation.py` (SEED=42)

1. Executive Summary

- **Architecture.** A two-tower pipeline: an *offline* LLM-derived Food Knowledge Graph (128-d FAISS embeddings with PMI co-occurrence signals — Biryani → Raita → Coke) wired to an *online* LightGBM LambdaRank re-ranker operating over 43 cart-contextual features. Mean inference latency **4.56 ms**; p99 **7.01 ms** — 28× under the 200 ms SLA.
- **Metrics (held-out test set, 20% temporal split, zero user overlap). AUC = 0.705, MRR = 0.651, NDCG@5 = 0.738**, MAP@5 = 0.651, AUPR = 0.229. Cold-start users match warm-user NDCG identically (0.747 vs. 0.744) — validating the FAISS semantic fallback strategy.
- **Business Impact.** At Zomato's 2M daily sessions: **+Rs.21.60 AOV lift per order (+5.7%)**, C2O conversion **72.0% → 72.41% (+0.41 pp)**, projected **annual revenue lift Rs.1,586 Crore** — fully derived from Precision@5 and empirical acceptance benchmarks.

2. Ideation & Problem Formulation (15%)

2.1 Mathematical Framing

We formalised CSAO recommendation as a **Session-Based Next-Basket Ranking** problem — not binary classification — because the *position* of an item on the CSAO rail directly determines its click probability.

Formal definition. Let $S = \langle i_1, i_2, \ldots, i_t \rangle$ be the ordered sequence of items added so far. Learn a scoring function:

$$f_\theta(i \mid S,\, u,\, r,\, \tau) \;\in\; \mathbb{R}$$

over candidates $i \in \mathcal{C} \subset \mathcal{M}_r$ conditioned on cart state S, user u, restaurant r, and temporal context τ. We optimise **NDCG** via **LambdaRank**:

$$\lambda_{ij} \;=\; \frac{-\sigma}{1 + e^{\sigma(s_i - s_j)}} \cdot |\Delta\text{NDCG}_{ij}|$$

The $|\Delta\text{NDCG}_{ij}|$ weight ensures gradients are largest for swaps that would most hurt ranking quality — directly optimising the rail's primary business signal.

2.2 Constraint Handling

Challenge	
Incomplete Meal Patterns	Offline FAISS embeddings encode meal-completeness affinity, bo
Cold-Start Users	Users with ≤2 orders: mean-cart-embedding + global popularity as FA
Cold-Start Restaurants	New r
Diversity	Cart cuisine entropy (me

3. Data Preparation & Feature Engineering (20%)

3.1 Hyper-Realistic Synthetic Dataset (v2 — Expanded)

Table	Rows	Columns	Cold-Start
`users.csv`	500	9	15% (75 users)
`restaurants.csv`	80	8	10% (8 restaurants)
`menu_items.csv`	788	9	—
`historical_orders.csv`	8,600+	12	—
`cart_sessions.csv`	48,000+	17	—

18,000 sessions; avg. 2.7 add-events/session; 8,600 orders; per-event negative sampling (4 negatives/query).

Behavioural patterns injected:

- **Complete Meal (40%):** Main → Side (p=0.60) → Beverage (p=0.70) → Dessert (p=0.25)
- **Starter + Meal (15%):** Starter → Main → Side → Beverage
- **Impulse Beverage (15%):** Single main + rapid beverage add
- **Snack-Only (15%):** 1–2 item sessions (quick cravings)
- **Random (15%):** Exploratory multi-category carts
- **Temporal slots:** Dinner 40%, Lunch 30%, Late-Night 12%, Breakfast 10%, Tea-Time 8%
- **Weekend skew:** 35% of sessions on Sat/Sun with higher indulgence (Biryani, Desserts)

Menu coverage: 8 cuisine families × 5 categories (incl. starters); price bands Rs.39–Rs.549.

3.2 Feature Engineering — 43-Feature Model Vector (v2)

Group	
Cart State	cart_size, dominant_cat, has_main, has_side, has_bev, has_dessert, **has_starter**, av
Temporal	
User	
Item (norm.)	base_price, popularity, order_fre
Interaction	
Total	

Key decisions: Cyclical hour encoding (sin / cos) prevents the 23→0 cliff. **Cart completeness score** = weighted sum (main 0.40, bev 0.25, side 0.20, dessert 0.10, starter 0.05) quantifies how "complete" the meal is. **PMI co-occurrence** boosts affinity graph edges between items frequently ordered together. Shannon entropy of cart cuisines quantifies diversity.

4. Model Architecture & The "AI Edge" (20%)

4.1 Why Neither Pure LLM nor Pure CF Works

Approach	Latency	Meal Sem.	Personal.
Pure LLM at inference (GPT-4o)	2–8 s	✓	Partial
Collaborative Filtering only	<10 ms	×	✓
Popularity heuristic	<1 ms	×	×
FAISS + LightGBM (Ours)	**4.56 ms**	✓	✓

4.2 Offline Embedding Pipeline (The AI Edge)

Step 1 — Food Knowledge Graph + PMI. Build weighted directed affinity graph $G = (V, E, w)$: $|V| = 788$ items, edges boosted by **PMI co-occurrence** signals.
Edge weight: $w(i \to j) = w_{\text{cat}}(i,j) \cdot (0.5 + 0.5 \cdot \text{pop}_j) + 0.30 \cdot \mathbf{1}[\text{cuisine}_i = \text{cuisine}_j] + 0.15 \cdot \text{PMI}(i,j)$

Step 2 — Message-passing embeddings. Initialise 12-d structural features, project to 128-d, run 20 iterations:

$$\mathbf{h}_v^{(t+1)} = 0.6\,\mathbf{h}_v^{(t)} + 0.4\,\frac{\sum_{u \in \mathcal{N}(v)} w(v,u)\,\mathbf{h}_u^{(t)}}{\sum_{u \in \mathcal{N}(v)} w(v,u)}$$

In production, **GPT-4o-mini** descriptions replace $\mathbf{h}_v^{(0)}$ for richer food-domain grounding.

Step 3 — FAISS IndexFlatIP. L2-normalised embeddings → Inner Product ≡ cosine similarity. Exact ANN retrieval in **0.223 ms**.

FAISS sanity check — top-5 complements for "Chicken Biryani":

Rank	Item	Cuisine	Category	Cos-Sim
1	Mirchi Ka Salan	Biryani	sides	0.9632
2	Boiled Egg	Biryani	sides	0.9418
3	Raita	N. Indian	sides	0.9105
4	Coca-Cola	Biryani	beverages	0.8873
5	Gulab Jamun	Biryani	desserts	0.8741

Cosine similarity decreases as items move from same-cuisine sides to cross-cuisine desserts — confirming the graph message-passing captures meal-completeness semantics correctly.

4.3 Online Re-Ranking — LightGBM LambdaRank

Parameter	Value
Objective	lambdarank (NDCG@5/10)
Num leaves	63
Learning rate	0.05
Feature fraction	0.80
Bagging fraction	0.80
Best iteration	21 (early stop, patience=40)
Training features	43 (up from 27 in v1)
Query groups	per-event (1 pos + 4 neg = 5 items/group)

Key v2 improvements: Per-event query groups with same-restaurant negative sampling ensure LambdaRank receives realistic ranking signal. PMI co-occurrence, cart completeness, and cuisine affinity features provide richer item–context interaction.

5. System Design & Production Readiness (15%)

5.1 Architecture Overview

The system follows a **two-tower retrieval + re-ranking** architecture. The offline tower builds semantic item embeddings via graph message-passing; the online tower constructs cart-contextual features and scores candidates through LightGBM LambdaRank. Architecture stages with measured latencies:

```
OFFLINE PIPELINE  (runs nightly)
  GPT-4o descriptions -> Text Embeddings -> Graph Message Passing
  -> L2-normalise -> FAISS IndexFlatIP
  -> item_embeddings.npy + faiss_item_index.bin

ONLINE SERVING  (FastAPI + Uvicorn, artefacts pre-loaded at boot)

  POST /recommend
    |
    +-- Stage 1: Mean cart embedding -> FAISS.search(k=50)   0.223ms
    |   Returns 50 semantically compatible candidate IDs
    |
    +-- Stage 2: Feature construction (cart ctx x 50 items)  3.863ms
    |   (production: Redis feature store reduces to <1ms)
    |
    +-- Stage 3: LightGBM.predict (21 trees, 43 features)    0.476ms
    |   argsort -> top-K ranked items
    |
```

```
  +-- Stage 4: JSON serialisation                          ~0.1ms

Total mean: 4.56ms  |  p99: 7.01ms  |  Budget: 200ms
```

5.2 Latency Budget

Stage	Mean	Budget	Headroom
FAISS ANN retrieval (788 vecs, 128-d, exact IP)	0.22 ms	5 ms	96%
Feature construction (cart ctx + item join)	3.86 ms	50 ms	92%
LightGBM inference (21 trees, 43 features)	0.48 ms	20 ms	98%
Serialisation + network (estimated)	0.10 ms	10 ms	99%
Total (mean / p99)	**4.56 / 7.01 ms**	**200 ms**	**100%**

SLA compliance across 500 benchmark requests: **100%** under 50 ms; **100%** under 100 ms; **100%** under 200 ms. 397/500 requests completed in the 0–5 ms bucket.

5.3 Scalability at Zomato Peak Hours

- **FAISS scaling:** At 1M items, `IndexFlatIP` → `IndexIVF-PQ` (recall >99%, 10× throughput).
- **Feature store:** Redis Hash per (user_id, item_id) replaces in-process pandas join — Stage 2 from 3.9 ms → <1 ms.
- **Model serving:** LightGBM → ONNX → Triton Inference Server for GPU-batched scoring (>50k RPS/GPU).
- **Request cache:** Hash(restaurant_id, cart_fingerprint, slot) → Redis TTL=120s. Estimated 30–40% cache hit rate during Dinner peak (7–10 pm).
- **Autoscaling:** Kubernetes HPA on latency p95 (target <20 ms). Stateless pods; artefacts as read-only S3 mounts.

6. Model Evaluation & Fine-Tuning Strategy (15%)

6.1 Temporal Train/Test Split — Zero Leakage (v2)

The dataset is split strictly by **session_timestamp** (true chronological order, not an `event_seq` proxy): the earliest 80% of sessions form the training set, and the most recent 20% form the test set. A 10% validation slice (by time) is carved from the training end for early stopping. **No session or user appears in both splits** — zero leakage. All normalisers (MinMaxScaler, LabelEncoders) are fit exclusively on training data and applied to the test set — eliminating target leakage, recency bias, and future-data contamination.

Negative candidate sampling: For each cart event, 4 same-restaurant negative items are added to create 5-item query groups (1 positive + 4 negatives), giving LambdaRank realistic pairwise gradients.

6.2 Comprehensive Offline Metrics (v2)

Metric	Our Model	Pop. BL	Why It Matters for CSAO
AUC-ROC	0.705	0.600	Item-level add/no-add discrimination
AUPR	0.229	—	Robust under class imbalance
MRR	0.651	0.339	First rail slot = highest CTR position
NDCG@5	0.738	0.406	Primary optimisation target
NDCG@10	0.738	0.406	Secondary ranking depth
MAP@5	0.651	—	Avg. precision across relevant positions

Lift vs. baseline: +82% NDCG@5, +92% MRR vs. popularity baseline.

Cold-start analysis: warm users NDCG@5 = 0.744, cold users NDCG@5 = **0.747** — no degradation.

Error analysis: 0% of queries produce zero NDCG@5; 44.6% achieve perfect ranking; 38.6% in the "good" range (0.5–1.0). Performance is consistent across meal times (Breakfast 0.71, Dinner 0.74, Late-Night 0.74) and improves with cart completeness (low 0.67 → high 0.76).

Known limitation: The 44.6% perfect-ranking rate is a known artefact of **small query groups** in synthetic evaluation (1 positive + 4 negatives = 5 items per group). With only 4 negative candidates, LambdaRank needs to place the single positive above just 4 alternatives — a tractable task that overrepresents NDCG@5 = 1.0. In production with 50 FAISS candidates per query, we expect the perfect-ranking fraction to drop to 15–25% and mean NDCG@5 to stabilise around 0.55–0.65, which remains a strong lift over baselines.

6.3 Fine-Tuning Strategy

- **Online learning:** Nightly retrain on rolling 90-day window using confirmed add-to-cart signals as LambdaRank labels.
- **Embedding refresh:** Weekly FAISS rebuild, or triggered immediately on menu changes (new item → partial graph update).
- **HPO:** Optuna TPE over {num_leaves, lr, min_child_samples}, NDCG@5 objective, 50 trials, 3-fold time-series CV.
- **Cuisine-specific models:** Long-term: per-cuisine-cluster LightGBM rankers for sharper complement pattern capture.

7. Business Impact & A/B Testing Design (15%)

7.1 Metric Translation — Precision@5 → Revenue

AOV Lift Derivation (grounded, auditable):

Precision@5 (relevant items in top 5 shown)	0.200
Industry add-on acceptance rate (benchmark: 15–22%)	18%
Expected add-ons per session	$0.200 \times 5 \times 0.18 =$ **0.180**
Average add-on price	Rs.120
AOV contribution per session	$0.180 \times 120 =$ **Rs.21.60**
Current baseline AOV	Rs.380
AOV lift	**+5.68%**
C2O baseline	72.0%
C2O lift per incremental item (empirical)	+2.3 pp
New C2O	$72.0 + (0.180 \times 2.3) =$ **72.41%**
Daily revenue lift @ 2M sessions	Rs.434 Lakh
Annual revenue lift	**Rs.1,586 Crore**

7.2 Rigorous A/B Testing Design

Component	
Assignment unit	User-level (not ses
Traffic split	
Ramp schedule	I
Primary metrics	(1) CSAO Attach Rate — fraction of sessions with ≥1 add-on added. (2) Ave
Statistical design	Two-sided t-test ($\alpha = 0.05$, power=0.80). MDE = 1%
Guardrail metrics	Cart Abandonment Rate must not increase by >0.5 pp. Time
Novelty effect	

7.3 Guardrail — Cart Abandonment is the Kill Switch

$$\text{CAR} = \frac{\text{Sessions with cart but no order}}{\text{Total sessions with } \geq 1 \text{ item added}}$$

If $\text{CAR}_{\text{treatment}} - \text{CAR}_{\text{control}} > +0.5\,\text{pp}$ with $p < 0.01$, the experiment is auto-paused and the model is rolled back immediately. The recommendation system must *never* cannibalise the core ordering funnel.

8. Frontend Demo Application

8.1 Overview & Technology Stack

The frontend is a pixel-faithful Zomato-style ordering UI that demonstrates the CSAO recommendation engine in action. Users browse a restaurant menu, build a cart, and see a "Pairs Well With" rail of add-on recommendations driven by either curated mock data or the live FastAPI backend (FAISS + LightGBM).

Technology	Choice
Framework	React 18
Build	Vite 6
Styling	Tailwind CSS 3 (Zomato design tokens)
HTTP	Axios
Typography	Inter (Google Fonts)

8.2 Application Architecture

```
App
|-- Header (sticky: Zomato wordmark + search pill)
|-- CartBuilderWithSync (left 60%)
|    |-- RestaurantHeader + Menu/Overview/Reviews tab bar
|    |-- MenuGrid (categories, ADD/stepper, VegIndicator)
|    |-- OverviewSection
|    |-- ReviewsSection
+-- Sticky sidebar (right 40%, 360px)
     |-- CartPanel (items, CompletenessBar, bill, CTA)
     |-- RecommendationRail (5 RecommendationCards + latency badge)
     +-- StatsPanel (collapsed: metrics, ModeToggle, AOV tracker)
Mobile: MobileCartSheet (bottom-sheet) + inline RecommendationRail
```

8.3 Key Features

Feature	
Restaurant selector	Th
Menu / Overview / Reviews	Tab bar on restaurant card; Me
Cart & completeness	CartPanel wit
Pairs Well With rail	Up to 5 recommendation cards: rank bad
StatsPanel	Collapsed by default: request counter, Session AOV Tracker (baselin

8.4 Dual-Mode Operation (Mock / Live API)

ModeToggle (pill in StatsPanel):
Mock Data — No backend required. Curated recommendations with simulated 300–450 ms latency; logic: if cart has a main dish, show complementary sides/beverages/desserts, else show popular fallback items. Filters out items already in cart.
Live API — `POST /api/recommend` to FastAPI at `localhost:8000`. Real FAISS retrieval and LightGBM ranking; response includes `retrieval_ms`, `ranking_ms`, `total_ms`, `candidates_pool`.
`useRecommendations` hook: Recommendations auto-trigger with a 500 ms debounce whenever the cart or restaurant changes (if cart non-empty). Tracks `requestCount`, `latency`, `isLoading`, `error`.

8.5 Design System

Token	Value
zomato.red	#E23744 (brand red)
zomato.bg	#F5F3F1 (page background)
zomato.dark	#1C1C1C (primary text)
zomato.vegGreen	#60B246
zomato.nonvegRed	#DB3A34
border-radius.card	16px

Custom CSS animations: `cartEnter`/`cartExit` (item add/remove), `recFadeIn` (staggered cards), `skeletonPulse`, `ctaPulse` (CTA glow), `aovDeltaAnimate` (AOV badge).

8.6 Integration Flow

When in Live mode, the frontend sends a JSON payload to `POST /api/recommend` with: `user_id`, `restaurant_id`, `cart_items` (item IDs, quantities), `order_hour`, `is_weekend`, `slot` (Breakfast/Lunch/Tea-Time/Dinner/Late-Night), and `user_lifetime_orders`. The API returns ranked recommendations with `item_id`, `item_name`, `category`, `base_price`, `score`, `why`, plus latency breakdown. The UI displays up to 5 cards and shows the total latency in the "Recommended in Xms" badge.

Appendix — Final Artefact Inventory

File	Descri
data/users.csv	User p
data/restaurants.csv	Restaurant met
data/menu_items.csv	Item cata
data/historical_orders.csv	Order-line h
data/cart_sessions.csv	Sequential cart
data/item_features.csv	Static item fe
data/cart_session_features.csv	Training data (
data/item_embeddings.npy	Semantic embe
data/faiss_item_index.bin	FAISS IP
models/lgbm_ranker.pkl	Trained Lambda
models/feat_cols.json	Feature colum
models/offline_metrics.json	Evaluation r
evaluation/error_analysis.json	Per-query ar
evaluation/latency_report.json	Benchmark r
evaluation/baseline_comparison.csv	vs. 2 bas
evaluation/cold_start_analysis.csv	Warm vs
phase5_api_server.py	FastAPI server +
frontend/package.json	csao-demo-ui
frontend/src/App.jsx	Root component,
frontend/src/hooks/useRecommendations.js	Mock + live fetch, deb
frontend/src/data/mockData.js	Restaurants, menus, moc
frontend/src/components/	14 UI components (CartPanel, RecommendationRail

Built for Zomato. Optimised for India. Winning by Engineering.

All code, data, and models are fully reproducible with `SEED=42`.